

Inception — 4-Day Study Guide

Day 1 — Docker Fundamentals: Dockerfiles, docker-compose.yml, Secrets & Volumes

What is Docker and why does it exist?

A **container** is a running process that is isolated from the rest of the system using Linux kernel features called **namespaces** (isolated view of filesystem, network, processes) and **cgroups** (CPU/memory limits). Unlike a VM which virtualises an entire OS with its own kernel, a container shares the host kernel — it is just a process with walls around it.

A **Docker image** is a read-only snapshot of a filesystem. When Docker runs an image, it adds a writable layer on top — that is your running container. Images are built from **Dockerfiles**.

Dockerfiles

A Dockerfile is a recipe. Each instruction creates a layer in the image.

```
FROM alpine:3.20          # base image — MUST be a specific version, not latest
RUN apk add --no-cache nginx openssl # install packages, --no-cache keeps image small
COPY ./tools/script.sh /usr/local/bin/ # copy files from host into image at build time
RUN chmod +x /usr/local/bin/script.sh # make script executable
EXPOSE 443                # documents which port the container listens on
CMD ["/usr/local/bin/script.sh"]     # default command when container starts
```

Rules the subject enforces on Dockerfiles

Rule	Why
FROM alpine:X.X.X — pinned version, not latest	Reproducibility: latest can change silently
No passwords in Dockerfiles	Dockerfiles go into git — passwords would be public
No tail -f, sleep infinity, while true in CMD/ENTRYPOINT	These are fake process-keeping hacks. Use exec to run the real process as PID 1
One Dockerfile per service	Separation of concerns, evaluators verify this
Image name must match service name	Evalsheet requirement

Why exec matters (PID 1)

When a container starts, the CMD/ENTRYPOINT becomes **PID 1**. PID 1 in Linux is special — it receives signals like SIGTERM when Docker stops the container. If your script launches nginx and then the script exits, nginx becomes an orphan. If your script runs nginx & sleep infinity, PID 1 is sleep — Docker signals the wrong process and the container hangs.

The correct pattern:

```
exec nginx -g 'daemon off;' # exec replaces the shell with nginx as PID 1
```

Now nginx IS PID 1. When Docker sends SIGTERM, nginx receives it and shuts down cleanly.

docker-compose.yml

docker-compose.yml orchestrates multiple containers. It defines services, networks, volumes, and secrets in one file.

Our file — annotated

```
networks:
  inception:                # creates a private virtual network called "inception"
    name: inception
    driver: bridge          # bridge = isolated from host, containers talk to each other

volumes:
  wordpress:                # named volume for WordPress files
    driver: local
    driver_opts:
      type: none
      o: bind
      device: /home/${USER}/data/wordpress # backed by this path on the host
  mariadb:                  # named volume for database files
    driver: local
    driver_opts:
      type: none
      o: bind
      device: /home/${USER}/data/mariadb

secrets:
  db_password:
    file: ./secrets/db_password.txt      # read from this file on the host
  wp_admin_password:
    file: ./secrets/wp_admin_password.txt
  wp_user_password:
    file: ./secrets/wp_user_password.txt

services:
  nginx:
    build: ./requirements/nginx          # build from this Dockerfile
    image: nginx:42                      # name the resulting image nginx:42
    container_name: nginx                # name the container nginx
    ports:
      - "443:443"                        # only nginx is exposed to the outside world
    depends_on:
      - wordpress                        # start wordpress first
    volumes:
      - wordpress:/var/www/html          # share WordPress files with nginx
    networks:
      - inception
    env_file:
      - .env                             # load non-sensitive variables
    restart: always                      # restart if container crashes

  wordpress:
    build: ./requirements/wordpress
    image: wordpress:42
    container_name: wordpress
    depends_on:
      - mariadb                          # start mariadb first
    volumes:
      - wordpress:/var/www/html          # WordPress and nginx share this volume
    env_file:
```

```

    - .env
secrets:
  - db_password                # injected as /run/secrets/db_password
  - wp_admin_password
  - wp_user_password
networks:
  - inception
restart: always

mariadb:
  build: ./requirements/mariadb
  image: mariadb:42
  container_name: mariadb
  volumes:
    - mariadb:/var/lib/mysql    # database files persist here
  env_file:
    - .env
  secrets:
    - db_password
  networks:
    - inception
  restart: always

```

Forbidden patterns

```

network: host    # FORBIDDEN – bypasses Docker networking, breaks isolation
links:           # FORBIDDEN – old deprecated feature, replaced by networks
--link           # FORBIDDEN – same, in Makefiles/scripts

```

Secrets vs Environment Variables

	Environment Variables (.env)	Docker Secrets
Storage	Plain text in .env file	Plain text in secrets/*.txt files
In git?	Must be gitignored	Must be gitignored
In container	Available as \$VAR	Available as a file at /run/secrets/<name>
Risk if leaked	Credentials exposed	Credentials exposed
Subject says	Use for non-sensitive config	“Strongly recommended” for passwords

How secrets appear inside a container:

```

# docker-compose.yml declares:
secrets:
  - db_password

```

```

# Inside the container, it appears as a file:
cat /run/secrets/db_password    # prints the password
db_pwd=$(cat /run/secrets/db_password)  # assign to variable in script

```

The .env file keeps non-sensitive variables: domain name, usernames, database name, WordPress title.

Volumes vs Bind Mounts

	Named Volumes (what we use)	Bind Mounts
Definition	Docker manages a directory, backed by a host path	Host path directly mounted
Portability	Can reference volume by name	Need full absolute path
In compose	volumes: section with driver_opts	Inline path like ./host:/container
Persistence	Data survives container deletion	Data survives container deletion
Eval check	docker volume inspect shows the host path	N/A

We use named volumes backed by bind mounts — this satisfies both requirements: volumes are named (visible in `docker volume ls`) AND data is at `/home/msuokas/data/` (verifiable in `docker volume inspect`).

Makefile

```
COMPOSE_FILE = srcs/docker-compose.yml
DATA_DIR = /home/$(USER)/data
SECRETS_DIR = srcs/secrets

all: build up                # default target

build: $(DATA_DIR)/mariadb $(DATA_DIR)/wordpress $(SECRETS_DIR)
    docker compose -f $(COMPOSE_FILE) build

$(DATA_DIR)/mariadb:          # prerequisite: create dir if missing
    mkdir -p $(DATA_DIR)/mariadb

$(DATA_DIR)/wordpress:
    mkdir -p $(DATA_DIR)/wordpress

$(SECRETS_DIR):              # prerequisite: create secrets dir if missing
    mkdir -p $(SECRETS_DIR)

up:
    docker compose -f $(COMPOSE_FILE) up -d

down:
    docker compose -f $(COMPOSE_FILE) down

clean:
    docker compose -f $(COMPOSE_FILE) down
    docker system prune -af    # remove all unused images

fclean: clean
    docker volume prune -f
    sudo rm -rf $(DATA_DIR)    # wipe host data

re: fclean all               # full rebuild from scratch
```

Key Makefile targets

Target	Does
<code>make</code> or <code>make all</code>	Build images + start containers
<code>make build</code>	Build images only
<code>make up</code>	Start already-built containers
<code>make down</code>	Stop and remove containers
<code>make clean</code>	Stop containers + prune images

Target	Does
make fclean	Full wipe including data and volumes
make re	fclean then all — complete fresh start

Day 2 — MariaDB: Configuration and Initialization Script

What is MariaDB?

MariaDB is an open-source relational database, a fork of MySQL. WordPress uses it to store all content: posts, users, settings, comments. It speaks SQL and listens on TCP port 3306.

The Dockerfile

```
FROM alpine:3.20

RUN apk add --no-cache mariadb mariadb-client && \
    mkdir -p /run/mysqld /var/log/mysql && \
    chown -R mysql:mysql /run/mysqld /var/log/mysql && \
    chmod 755 /run/mysqld
```

Why these steps at build time (not in the script)?

The directories `/run/mysqld` (for the socket file) and `/var/log/mysql` (for logs) must exist before MariaDB starts. Doing it in the Dockerfile means it happens once during image build, not on every container start. The `chown mysql:mysql` is required because MariaDB runs as the `mysql` user — if it can't write to its socket directory, it crashes.

```
COPY ./tools/50-server.cnf /etc/my.cnf.d/ # database configuration
COPY ./tools/script.sh /
RUN chmod +x /script.sh
CMD ["/script.sh"]
```

The server configuration: 50-server.cnf

```
[server]

[mysqld]
user                = mysql          # run as the mysql system user
pid-file            = /run/mysqld/mysqld.pid
socket              = /run/mysqld/mysqld.sock
port                = 3306
datadir             = /var/lib/mysql  # where database files are stored
bind-address        = 0.0.0.0         # CRITICAL: listen on all interfaces
log_error           = /var/log/mysql/error.log
character-set-server = utf8mb4        # full Unicode including emoji
collation-server    = utf8mb4_general_ci
```

Why `bind-address = 0.0.0.0` is critical

By default MariaDB only listens on `127.0.0.1` (localhost). Inside Docker, WordPress is a different container with a different IP. Without `bind-address = 0.0.0.0`, WordPress's connection attempt to `mariadb:3306` would be

refused — the database is listening on the wrong interface. Setting it to `0.0.0.0` tells MariaDB to accept connections on any network interface including the Docker bridge network.

Why `query_cache_size` is NOT in the config

`query_cache_size` was removed in MariaDB 10.6+. If you include it, MariaDB prints an error and may crash on startup. Always check the MariaDB version's changelog when copying config from older examples.

The initialization script: `script.sh`

```
#!/bin/sh
set -e                                # exit immediately on any error

if [ ! -e /var/lib/mysql/.firstmount ]; then
    db_pwd=$(cat /run/secrets/db_password) # read password from secret file
```

Why the `firstmount` guard?

`/var/lib/mysql` is a volume — it persists across container restarts. Without this guard, `mysql_install_db` would re-initialize an already-initialized database on every restart, destroying all data. The `.firstmount` flag file lives inside the volume, so it persists with the data.

```
mysql_install_db \
  --datadir=/var/lib/mysql \
  --skip-test-db \
  --user=mysql \
  --group=mysql \
  --auth-root-authentication-method=socket \
  >/dev/null 2>&1
```

`mysql_install_db` creates the initial database file structure — the `mysql` system tables that MariaDB needs to track users, permissions, and databases. `--skip-test-db` skips creating the useless test database. `--auth-root-authentication-method=socket` means the root user authenticates via Unix socket (no password needed for root when running as the `mysql` OS user) — this is the Alpine/MariaDB default and avoids the need for a root password.

```
mysqld --bootstrap --user=mysql >/dev/null 2>&1 << EOF
CREATE DATABASE IF NOT EXISTS $db_name;
CREATE USER IF NOT EXISTS '$db_user'@'%' IDENTIFIED BY '$db_pwd';
GRANT ALL PRIVILEGES ON $db_name.* TO '$db_user'@'%';
FLUSH PRIVILEGES;
EOF
```

What is `--bootstrap` and why is it the right approach?

`mysqld --bootstrap` starts MariaDB in a special single-user initialization mode. It reads SQL commands from stdin, executes them, and exits. No TCP port is opened, no background process is created. This is the cleanest way to run initialization SQL without starting a background server.

The alternative (starting `mysqld_safe` &, waiting, running SQL, killing it) is fragile: you need a sleep or wait loop, and there is a background process in the entrypoint — the evaluator may question this.

The SQL itself: `CREATE DATABASE IF NOT EXISTS` — idempotent, safe to run multiple times - `CREATE USER '$db_user'@'%'` — the `%` wildcard allows connections from any host (required because WordPress connects from a different container IP) - `GRANT ALL PRIVILEGES ON $db_name.*` — WordPress needs full access to its own database - `FLUSH PRIVILEGES` — makes the grants take effect immediately

```
touch /var/lib/mysql/.firstmount    # set the flag so this block never runs again
fi
```

```
exec mysqld_safe                                # start the real server as PID 1
```

mysqld_safe is a wrapper around mysqld that adds crash-recovery: if MariaDB dies, mysqld_safe restarts it. The exec replaces the shell with mysqld_safe as PID 1.

Why log into MariaDB during evaluation

```
docker exec -it mariadb mariadb -u ss -p
# password from srcs/secrets/db_password.txt
show databases;    # shows lol1
use lol1;
show tables;       # shows all WordPress tables
```

This proves the database is initialized, the user exists, and WordPress has written data.

Day 3 — NGINX: SSL/TLS and Reverse Proxy

What does NGINX do in this project?

NGINX is the only container exposed to the outside world (port 443). It has two jobs:

1. **Terminate TLS** — handle the HTTPS encryption/decryption
2. **Reverse proxy** — forward PHP requests to WordPress/php-fpm on port 9000

All HTTP traffic flows: Browser → NGINX (443 TLS) → WordPress (9000 FastCGI) → MariaDB (3306)

The Dockerfile

```
FROM alpine:3.20

RUN apk add --no-cache nginx openssl && \
    mkdir -p /etc/nginx/ssl              # create SSL directory at build time

COPY ./tools/script.sh /usr/local/bin/
RUN chmod +x /usr/local/bin/script.sh

EXPOSE 443                               # only port 443 – no HTTP
CMD ["/usr/local/bin/script.sh"]
```

The SSL directory is created at build time so the script doesn't need to — and the directory is guaranteed to exist before openssl tries to write certificates there.

The entrypoint script: script.sh

```
#!/bin/sh
set -e

if [ ! -e /etc/.firstrun ]; then
```

The firstrun guard: NGINX stores the SSL certificate in the container's filesystem (not a volume), so on a fresh container it would regenerate. But if you mount a config volume, the guard prevents double-generation. It also means the domain name is baked in once.

```
openssl req -x509 -days 365 -newkey rsa:2048 -nodes \
-out /etc/nginx/ssl/cert.crt \
-keyout /etc/nginx/ssl/cert.key \
-subj "/CN=$DOMAIN_NAME" \
>/dev/null 2>&1
```

Generates a **self-signed certificate**: - -x509 — self-signed (no Certificate Authority) - -days 365 — valid for one year - -newkey rsa:2048 — generate a new 2048-bit RSA key pair - -nodes — no passphrase on the private key (needed because nginx reads it unattended at startup) - CN=\$DOMAIN_NAME — Common Name matches the domain (e.g., msuokas.42.fr)

```
chmod 600 /etc/nginx/ssl/cert.key # private key: owner read only
chmod 644 /etc/nginx/ssl/cert.crt # certificate: readable by all
```

Security: the private key must never be world-readable. If an attacker gets the private key, they can decrypt all HTTPS traffic.

```
cat > /etc/nginx/http.d/default.conf << EOF
server {
    listen 443 ssl http2;
    listen [::]:443 ssl http2;          # IPv4 and IPv6
    server_name $DOMAIN_NAME;

    ssl_certificate /etc/nginx/ssl/cert.crt;
    ssl_certificate_key /etc/nginx/ssl/cert.key;
    ssl_protocols TLSv1.2 TLSv1.3;      # subject requirement: ONLY these two
    ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:...; # strong cipher list
```

The cipher list enforces forward secrecy (ECDHE) and modern authenticated encryption (GCM). Older weak ciphers (RC4, DES, CBC-SHA) are excluded.

```
client_max_body_size 64M;          # allow large uploads (images, themes)

add_header X-Frame-Options "SAMEORIGIN" always;    # prevent clickjacking
add_header X-Content-Type-Options "nosniff" always; # prevent MIME sniffing
add_header X-XSS-Protection "1; mode=block" always; # legacy XSS filter

root /var/www/html;                # WordPress files are mounted here
index index.php index.html index.htm;

location / {
    try_files $uri $uri/ /index.php?$args;
}
```

try_files is essential for WordPress permalinks. When you visit /about/, NGINX first checks if /about/ is a real file, then a real directory, then falls back to index.php?args=about/ — which is how WordPress's router handles URLs.

```
location ~ [^/]\.php(/|$) {
    try_files $fastcgi_script_name =404; # security: 404 if PHP file missing
    fastcgi_pass wordpress:9000;        # send to WordPress container
    fastcgi_index index.php;
    fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
    fastcgi_param PATH_INFO $fastcgi_path_info;
    fastcgi_split_path_info ^(.+\.php)(/.*)$;
    include fastcgi_params;
}

location ~ /\.ht {
    deny all;                          # block access to .htaccess files
}
EOF
touch /etc/.firstrun
```



```
fi

nginx -t                                # test config syntax before starting
exec nginx -g 'daemon off;'             # run nginx in foreground as PID 1
```

Why nginx -g 'daemon off;'?

By default, nginx forks into the background and the foreground process exits. In a container, when PID 1 exits, the container stops. `daemon off;` keeps nginx in the foreground.

Why fastcgi_pass wordpress:9000 works

Docker's built-in DNS resolves container names within the same network. Because both nginx and wordpress are on the inception network, nginx can reach wordpress by hostname. Port 9000 is php-fpm's listening port.

What the evaluator checks for NGINX

```
# TLS version
openssl s_client -connect msuokas.42.fr:443 2>/dev/null | grep "Protocol"

# Can't connect on HTTP
curl http://msuokas.42.fr           # should fail

# Can connect on HTTPS
curl -k https://msuokas.42.fr      # should return HTML
```

Day 4 — WordPress: php-fpm, WP-CLI, and Installation

What is php-fpm?

PHP-FPM (FastCGI Process Manager) is a way to run PHP as a separate service rather than embedded in a web server. NGINX speaks FastCGI protocol to php-fpm: it sends the PHP filename and request parameters, php-fpm executes the PHP code and returns HTML.

This is why there are two containers: NGINX handles HTTP/TLS, WordPress/php-fpm handles PHP execution. They share a volume so NGINX can find the PHP files.

The Dockerfile

```
FROM alpine:3.20

RUN apk add --no-cache \
    php83 php83-fpm php83-mysqli php83-pdo_mysql \    # PHP + database drivers
    php83-json php83-mbstring php83-xml php83-dom \    # data parsing
    php83-simplexml php83-tokenizer php83-curl \        # HTTP and parsing
    php83-openssl php83-zip php83-fileinfo php83-iconv \
    php83-gd php83-intl \                               # image processing, i18n
    mariadb-client curl                                 # for mariadb-admin ping, WP-CLI download
```

Each php83 extension is a separate package on Alpine. WordPress and its plugins need all of these — missing even one causes a white screen of death or broken features.

```

WORKDIR /var/www/html          # all subsequent commands run here

COPY ./conf/www.conf /etc/php83/php-fpm.d/www.conf # pool configuration
COPY ./tools/script.sh /usr/local/bin/
RUN chmod +x /usr/local/bin/script.sh

EXPOSE 9000
CMD ["/usr/local/bin/script.sh"]

```

The php-fpm pool config: www.conf

```

[www]
user = nobody          # run php-fpm workers as the nobody user
group = nobody
listen = 9000          # listen on TCP port 9000 (not a unix socket)
pm = dynamic           # dynamic process management
pm.max_children = 5    # max concurrent PHP processes
pm.start_servers = 2   # start with 2 workers
pm.min_spare_servers = 1 # minimum idle workers
pm.max_spare_servers = 3 # maximum idle workers

```

Why nobody instead of www-data?

Alpine Linux does not have a `www-data` user by default (that is a Debian/Ubuntu convention). The `nobody` user exists on Alpine and is the standard unprivileged user. Using an unprivileged user means a compromised PHP process cannot write to system files.

Why copy the config instead of editing it in the script?

Copying at build time is cleaner: the final config is in version control, visible and auditable. Editing with `sed` in the entrypoint script means the “real” config is hidden in string manipulation.

The installation script: script.sh

```

#!/bin/sh
set -e

if [ ! -e .firstmount ]; then
    db_pwd=$(cat /run/secrets/db_password)
    wp_admin_pwd=$(cat /run/secrets/wp_admin_password)
    wp_user_pwd=$(cat /run/secrets/wp_user_password)

```

Reading all passwords from secret files at the very start. They are stored in shell variables for use below — they are never written to any file or logged.

```

    mariadb-admin ping --protocol=tcp --host=mariadb \
        -u "$db_user" --password="$db_pwd" --wait >/dev/null 2>&1

```

Race condition prevention. Docker’s `depends_on` only waits for the container to START, not for the service inside it to be ready. MariaDB takes a few seconds to initialize after the container starts. Without this wait, WordPress would try to connect to MariaDB before it is ready and fail. `mariadb-admin ping --wait` keeps retrying until MariaDB accepts connections.

```

    if [ ! -f wp-config.php ]; then
        wp core download --allow-root

```

WP-CLI downloads WordPress core files into `/var/www/html` (the `WORKDIR`). `--allow-root` is needed because the script runs as root inside the container.

```
wp config create --allow-root \
  --dbhost=mariadb \
  --dbuser="$db_user" \
  --dbpass="$db_pwd" \
  --dbname="$db_name"
```

Creates wp-config.php — WordPress's main configuration file. dbhost=mariadb uses the container name, which Docker DNS resolves to the MariaDB container's IP on the inception network.

```
wp core install --allow-root \
  --skip-email \
  --url="$DOMAIN_NAME" \
  --title="$WP_TITLE" \
  --admin_user="$WP_ADMIN_USR" \
  --admin_password="$wp_admin_pwd" \
  --admin_email="$WP_ADMIN_EMAIL"
```

Installs WordPress: creates all database tables, creates the admin user, sets the site URL. --skip-email skips sending a welcome email. After this command, WordPress is fully installed — no web installer needed.

The admin username must NOT contain “admin”, “Admin”, “administrator”, etc. (evalsheet requirement). Using forstman1 passes this check.

```
if ! wp user get "$WP_USR" --allow-root >/dev/null 2>&1; then
  wp user create "$WP_USR" "$WP_EMAIL" \
    --role=author --user_pass="$wp_user_pwd" --allow-root
fi
```

Creates the second (non-admin) user. The if guard checks if the user already exists to make the operation idempotent. Role author can write posts and comments but cannot access admin settings.

The evalsheet requires two users: one admin (the one above) and one non-admin. Both must exist in the database.

```
wp theme install astra --activate --allow-root # install a theme
wp plugin update --all --allow-root           # update all plugins
else
  echo "WordPress already installed."
fi

chmod -R o+w /var/www/html/wp-content # allow web uploads
touch .firstmount                      # never re-install
fi

exec /usr/sbin/php-fpm83 -F             # start php-fpm in foreground as PID 1

-F flag keeps php-fpm in the foreground (equivalent to daemon off for nginx). exec makes it PID 1.
```

Why WP-CLI instead of the web installer?

The web installer requires manual browser interaction. WP-CLI automates the entire installation from the command line — the container self-configures on first start, no human steps needed. This is the professional approach and what the project expects.

Verifying WordPress works during evaluation

```
# Check the container is running
docker compose -f srcs/docker-compose.yml ps wordpress

# Check php-fpm is listening
docker exec -it wordpress netstat -tlnp | grep 9000
```

```
# Check the volume
docker volume inspect wordpress

# Verify two users in the database
docker exec -it mariadb mariadb -u ss -p lol1 \
-e "SELECT user_login, user_email, user_registered FROM wp_users;"

# Check WordPress files are on the host
ls /home/msuokas/data/wordpress/
```

Common failure modes and fixes

Problem	Cause	Fix
WordPress shows installer page	Script didn't run, or wp-config.php missing	Check container logs: docker logs wordpress
Database connection error	MariaDB not ready, wrong credentials, or bind-address missing	Check 50-server.cnf has bind-address = 0.0.0.0
php-fpm 502 Bad Gateway	WordPress container crashed or wrong port	Check docker ps, check www.conf has listen = 9000
Content gone after restart	.firstmount guard missing, volume not mounted	Check docker volume inspect wordpress
Admin username rejected	Contains "admin"	Use a username like forstman1

Summary — How everything connects

Browser

| HTTPS port 443
v

[NGINX container]

- Terminates TLS (self-signed cert)
- Serves static files from /var/www/html (shared volume)
- FastCGI proxy for *.php requests
- | FastCGI port 9000
v

[WordPress container]

- php-fpm executes PHP code
- Reads/writes WordPress files in /var/www/html (shared volume)
- | TCP port 3306
v

[MariaDB container]

- Stores all WordPress data (posts, users, settings)
- Files persist at /home/msuokas/data/mariadb (bind-mounted volume)

All three containers are on the inception Docker network. Only NGINX has a port exposed to the host. MariaDB and WordPress are completely internal — they cannot be reached from outside the Docker network.